

졸업 프로젝트 8월 2차 격주보고서		
SNN을 활용한 고효율 추천 시스템 개발 및 최적화 연구		
2020058259 이승현 2020061676 이강혁		지도교수 김상욱 교수님
지난 2주간 진행사항	- ANN to SNN 변환을 위한 ANN 선행 작업	△
진행 내용	ANN to SNN 변환을 위한 ANN 선행 작업 기존 ANN의 연속적인 활성화 값을 스파이킹 뉴런의 시간당 평균 스파이크 발사 빈도로 변환하여 SNN을 근사적으로 구현하는 방법을 다음과 같이 구현하였습니다. 1. Rate Encoding ANN의 특정 레이어를 통과한 결과값을 SNN의 입력 스파이크로 바꿔줍니다. ANN의 뉴런 활성화 값이 크면 클수록, SNN의 해당 뉴런이 더 빈번하게 스파이크를 발생시키도록 매핑합니다. 코드에서는 ANN의 GMF와 MLP 경로에서 나온 특징 벡터를 가져와 각 원소 값을 Spike Source Poisson의 rate 파라미터로 사용했습니다. 예를 들어 특징 벡터의 어떤 값이 80.5라면 그에 해당하는 SNN 입력 뉴런은 1초동안 평균적으로 약 80~81회의 스파이크를 생성합니다. 2. 가중치 직접 매핑 SNN의 시냅스 가중치는 별도의 학습 없이, 미리 학습된 ANN의 가중치를 그대로 가져와 사용합니다. 입력 뉴런의 발화율이 ANN의 활성화 값에 비례하므로 동일한 가중치를 사용하면 출력 뉴런의 발화율 또한 ANN의 최종 출력 값에 비례할 것이라는 가정에 기반합니다. 코드에서 model.output.weight (ANN의 마지막 Linear Layer 가중치)를 직접 가져와 SNN의 시냅스 연결 가중치로 설정했습니다.	

	3. 스파이크 횟수 디코딩 SNN의 최종 출력은 시간에 따른 스파이크의 나열이므로, 이를 다시 ANN의 출력처럼 하나의 숫자로 해석하는 과정이 필요합니다. 가장 간단하고 직관적인 방식으로, 정해진 시뮬레이션 시간 동안 출력 뉴런에서 발생한 총 스파이크 횟수를 최종 점수로 간주합니다. 코드의 마지막 부분에서 <code>len(st_out)</code>을 통해 출력 뉴런의 총 스파이크 개수를 계산하여 최종 추천 점수로 사용했습니다. 스파이크가 많이 발생했다는 것은 그만큼 강한 신호를 받았다는 의미이며, 이는 ANN의 높은 출력 점수에 해당합니다.
진행 중 이슈	NCF와 SNN의 추론에서의 에너지 사용량과 수행 시간을 비교한 결과 NCF의 결과가 더 좋았습니다. 이에 대한 원인을 찾고 수정하고자 합니다. 1. 에너지 효율 비교 이번 실험에서 NCF(ANN)가 SNN보다 에너지 효율이 높게 나온 이유로 크게 시뮬레이션 오버헤드와 계산 모델과 문제 유형의 부조화 두 가지를 뽑았습니다. (1) 시뮬레이션 오버헤드 -> SNN 추론 과정에 포함된 계산 외의 부가 과정으로 인해 생긴 오버헤드입니다. NCF는 학습된 모델에 값을 넣어 바로 답을 얻는 직접적인 계산인 반면, 이번 실험의 SNN은 전문 하드웨어(SpiNNaker)를 사용하는 시뮬레이션입니다. 따라서 실제 계산(Runtime)은 0.1초에 불과했지만, 모델을 하드웨어에 맞게 변환하고(매핑), 데이터를 전송하고(로딩), 결과를 다시 가져오는(추출) 등의 준비 및 정리 과정에 많은 에너지가 소모되었습니다. 오버헤드를 포함한 전체 에너지 소비량을 동일한 단위(mWh)로 변환하여 비교해 보았습니다. • NCF 소비 전력: 0.000220 mWh • SNN 소비 전력: 0.003339 kWh = 3,339 mWh 여기서 오버헤드를 제거하고 계산에 사용된 에너지만 비교해보았습니다.

- **NCF 소비 전력 (계산):** 0.000220 mWh
- **SNN 소비 전력 (계산):** 1.917 Joules $\approx$ 0.5325 mWh

오버헤드를 제외하고 비교했을 때, 전체 에너지 소비량보다 분명하게 나아졌지만, 여전히 NCF가 훨씬 효율적인 것을 볼 수 있습니다. 이에 대한 이유는 하나의 추천만 정적으로 추론하였기 때문입니다.

(2) 계산 모델과 문제 유형의 부조화

NCF와 SNN의 특징은 다음과 같습니다.

- **NCF의 특징:** NCF 추천은 정적인(static) 입력(사용자 ID, 아이템 ID)에 대해 한 번의 연산(Forward Pass)으로 정적인 출력(추천 점수)을 내는 작업입니다. 이는 그래픽 카드를 사용한 행렬 곱셈을 기반으로 정확하고 빠르게 결과를 도출합니다.
- **SNN의 특징:** SNN과 뉴로모픽 하드웨어는 시간에 따라 데이터가 변화하고 이벤트가 발생하는 시간적(temporal)이거나 이벤트 기반(event-based)인 문제에서 강점을 보입니다. 연속적으로 들어오는 신호를 처리할 때, 변화가 있을 때만 스파이크를 발생시켜 에너지를 절약합니다.

SNN은 시간의 흐름에서 데이터의 변화가 생길 때, 해당 부분만 추가적으로 수정하여 에너지를 절약하는 방식이고, 이 부분의 SNN의 핵심 아이디어입니다. 저희가 실행한 테스트는 시간과 데이터의 변화를 고려하지 않고 한 번의 추천만 제공하므로 SNN에게 적합하지 않았습니다.

(3) 인코딩 & 디코딩

인코딩은 Rate Encoding, 디코딩은 Spike Count Decoding을 사용하였습니다. 이는 값의 크기를 spike의 빈도 수로 직접적으로 변환하는 방식으로 단순하고 직관적입니다. 하지만, 강한 신호를 표현하기 위해 수많은 스파이크를 생성해야 하므로, SNN의 가장 큰 장점인 sparsity를 저해합니다. SNN에서는 스파이크 하나하나가 에너지 소모이므로 에너지 소모량을 증가시킵니다. 또한, 발화를 결정할 때 spike가 모일 충분한 시간을 기다려야하고, 추론 결과를 나타내기 위한 디코딩에서 spike의 수를 세서 값으로 변환해야 하는 것에 시간이 소요됩니다.

이를 해결하기 위해 Temporal Coding 방식을 도입하고자 합니다. 대표적으로 TTFS(Time-to-First-Spike) 인코딩은 신호가 강할수록 하나의

스파이크를 더 빠르게 보내는 방식입니다. 이는 훨씬 적은 스파이크를 사용하면서도 더 빠른 시간 안에 정보를 전달할 수 있습니다.

2. 추론 시간 비교

(1) 시뮬레이션 오버헤드 & 계산 모델과 문제 유형의 부조화

NCF는 0.006초가 걸렸고, SNN은 전체 과정에 81.6초가 걸렸습니다. 이는 에너지 효율 문제와 같은 시뮬레이션 오버헤드 때문입니다.

SNN의 전체 추론 시간은 아래와 같습니다.

1. **매핑 (~34.2초)**: 정의된 SNN 모델을 SpiNNaker 보드의 어느 칩에, 어떻게 배치할지 계산하는 과정입니다.
2. **데이터 로딩 (~24.9초)**: 모델의 가중치, 연결 정보, 입력 스파이크 데이터 등을 PC에서 SpiNNaker 보드로 전송하는 과정입니다.
3. **실제 실행 (0.1초)**: `pynn.run(100.0)`에 해당하는 부분으로, 실제 스파이킹 계산이 일어나는 시간입니다.
4. **결과 추출 (~22.5초)**: 시뮬레이션이 끝난 후, 출력 스파이크 같은 결과 데이터를 보드에서 다시 PC로 가져오는 과정입니다.

결론적으로 SNN 추론 시간의 대부분이 실제 계산이 아닌, 시뮬레이션 환경을 준비하고 결과를 정리하는 오버헤드에 사용되었습니다. 또한, 그렇기 때문에 SNN은 일회성 작업보다는, 지속적으로 입력 데이터가 들어오는 실시간 스트리밍 환경에서 오버헤드 없이 계산만 연속적으로 수행할 때 효율을 발휘합니다.

(2) 실행 시간 설정 - `pynn.run(100.0)`

SNN에서는 실행시간과 timestep 개념이 하이퍼파라미터로 설정됩니다. 이는 epoch과 비슷한 개념으로, 실행시간을 timestep으로 나눈 값이 epoch와 동일하게 동작합니다. 이때, 실행시간은 실제 모델이 계산하는 시간이 됩니다. 즉, 사용자가 설정하는 하이퍼파라미터, 일종의 epoch가 곧바로 수행시간이 된다고 할 수 있습니다. 저희가 진행한 테스트에서는 실행시간을 100ms으로 설정하였고, 이는 그대로 모델의 계산 시간이 된 것을 확인할 수 있었습니다.

	3. 해결방안 NCF보다 SNN이 더 에너지 효율적으로 동작하게 하려면 SNN의 핵심 장점인 시간 기반 및 이벤트 기반 처리와 데이터 희소성을 적극적으로 활용하도록 문제 자체와 모델 접근 방식을 바꿔야 합니다. 현재의 일회성 추론 방식은 SNN의 약점을 부각하고 장점을 완전히 무효화합니다. SNN의 효율을 발휘하기 위한 해결 방안은 다음과 같습니다. 문제 자체를 SNN에 적합하게 변경 이는 근본적인 해결방법으로, '정적인 추천 점수 계산' 문제를 '실시간 사용자 행동에 반응하는 동적인 추천' 문제로 재정의하는 것입니다. 이벤트 기반 작동: 사용자가 아무 행동도 하지 않을 때는 SNN이 거의 에너지를 소모하지 않습니다. 클릭이나 탐색 같은 이벤트가 발생할 때만 해당 정보를 처리하기 위해 스파이크가 발생하고 계산이 수행됩니다. 시간 정보 활용: 사용자의 행동 순서나 시간 간격 같은 시간적 정보를 자연스럽게 모델에 통합할 수 있습니다. 오버헤드 제거: 모델을 한 번만 뉴로모픽 하드웨어에 로드해두고, 지속적으로 들어오는 이벤트 스트림을 처리하면 되므로 매번 높은 시뮬레이션 준비/정리 비용이 발생하지 않습니다.
다음 2주간 계획	- ANN to SNN 변환을 위한 ANN 데이터셋 선행 작업 - SNN 모델 인코딩/디코딩 알고리즘 수정